

Towards Transparent Checkpointing with AI-driven Code Generation

Abstract—Adding reliable checkpoint/restart support to an MPI scientific application is a time-consuming expert effort that requires deep knowledge of both the application and resilience. We ask whether a frontier large language model can perform this work end-to-end without human intervention. We assemble a benchmark suite of MPI applications spanning diverse domains and computation patterns, and drive an iterative code-generation loop for each application using Anthropic’s Claude Opus 4.7 invoked through the OpenCode CLI. Across seventeen scientific applications, the LLM generates working checkpoint/restart code in roughly 45 minutes on average while consuming 5.1 M tokens per application. The generated code adds statistically zero overhead during normal failure-free execution on fourteen of seventeen applications ($\pm 2\%$ of original execution time), and recovers from injected process failures with efficiency comparable to human-engineered checkpoint/restart implementations. These results suggest that automated end-to-end LLM-driven resilience engineering is technically viable today for a meaningful fraction of HPC applications.

Index Terms—checkpoint/restart, large language models, resilience, HPC, fault tolerance

I. INTRODUCTION

Modern scientific computing combines high performance computing (HPC) and artificial intelligence (AI) in complex workflows that span heterogeneous infrastructure, from super-computing centers to scientific instruments. This shift unlocks heterogeneous accelerators and elastic capacity, but also fundamentally reshapes the failure landscape: long-running MPI simulations that once executed on a single homogeneous batch allocation now span resources whose mean-time-between-failure (MTBF) varies significantly; fluctuates dynamically as nodes are preempted, evicted, or reassigned under multi-tenant pressure; and is largely unpredictable at job-submission time [1], [2]. Under such circumstances, checkpoint/restart, the dominant resilience mechanism in HPC, faces several important challenges: it is increasingly difficult to identify: (1) which components of the workflow are tightly coupled in ways that let failures propagate rapidly among distributed processes; (2) which data structures within those components are critical and must be protected, and (3) at what point in the code it is safe to checkpoint those data structures. These challenges directly impact the correctness, performance and scalability of checkpoint/restart solutions used to enable resilient execution.

Manual coding of checkpoint/restart solutions to address these challenges can be tedious due to the need to combine performance modeling; deep understanding of interactions under parallelism to maintain a consistent global state at scale; fine-tuning; and debugging. Recent advances in large language models (LLMs) and tool-using coding agents [3], whose

performance on bounded software-engineering benchmarks now approaches that of professional developers [?] and whose agentic workflows can autonomously read source trees, plan multi-step transformations, and react to validator feedback, can offer an alternative that is both cheaper to implement and potentially more efficient. The central question of this paper is: *can a frontier LLM coding agent be driven, end-to-end and with no human in the loop, to add production-grade checkpoint/restart resilience to an unmodified MPI scientific application, and at what cost relative to the resulting correctness, performance and scalability?*

Key Insights and Contributions. This paper investigates the operational viability of *LLM-driven resilience engineering* as a building block for flexible scientific infrastructures. The observation we exploit is that the cognitive work of adding checkpoint/restart (identifying the data structures that make up the critical state, the correct place in the code where to checkpoint it, guarantee global consistency at scale, rebuild other data structures on restart based on the critical state) involves structured reasoning that modern coding agents are well suited for. The trade-off space, however, is far from understood: every iteration of the agent costs millions of tokens and tens of minutes of wall time; conservative state-coverage decisions inflate checkpoint footprint and erode scalability; a misidentified place in the code where it is safe to checkpoint silently corrupts a restart. Our goal is two-fold: (1) establish empirically that frontier LLM coding agents can produce correct, performant resilience code with no human intervention; and (2) characterize the cost, correctness and performance trade-offs that determine when this approach is practical for per-deployment resilience synthesis on dynamically provisioned, heterogeneous infrastructures. We summarize our contributions as follows:

- 1) **End-to-end LLM-driven resilience pipeline:** We design a closed-loop generate–validate–repair pipeline that drives a frontier coding agent to add VeloC-based checkpoint/restart to an unmodified MPI source tree with no human in the loop. The validator builds the generated code, runs it failure-free, injects a single-rank fault, and structurally checks that the application restarts correctly and reproduces the vanilla output (§ IV).
- 2) **Reusable resilience benchmark suite:** We assemble seventeen MPI applications covering diverse domains, code sizes (1K to 627K LOC), state structures, and checkpoint back-ends. Each is shipped with a resilience-stripped *vanilla* source (given to the agent) and an upstream-checkpointed *reference* (human-engineered baseline) (§ III).

3) **Empirical characterization of the cost, correctness and performance/scalability trade-off:** We quantify all three axes across the suite. On *cost*, the agent converges in a median of three iterations, averaging 5.1 M tokens and 45 minutes of loop wall time per application. On *correctness*, it consistently identifies a small, sufficient critical-state set and globally safe checkpoint points on every application. On *performance*, the generated code imposes statistically zero failure-free overhead on fourteen of seventeen apps and recovers from injected faults with efficiency comparable to the upstream human-engineered reference (§ V).

II. RELATED WORK

Application-level checkpointing approaches such as VeLoC [4], FTI [5], and SCR [6] require developers to manually declare critical state, place checkpoint calls at safe points, and wire restart logic into initialization paths. By saving only the state needed for recovery, they keep checkpoints small and reduce both runtime and storage overhead. These systems typically rely on *multi-level resilience strategies* [6], [7], where checkpoints are staged to a fast node-local tier, such as RAM, for recovery from frequent soft failures, then asynchronously flushed to a slower shared tier, such as a parallel file system, to survive critical failures. A rich body of work further optimizes this pipeline: asynchronous flush engines overlap checkpoint I/O with application compute [8], [9]; write aggregation coalesces per-rank writes into stripe-aligned bulk transfers [10]; differential and incremental checkpointing persist only changed bytes [11], [12]; and compression or lossy encoding reduces footprint at modest reconstruction cost [13], [14]. Checkpoint cadence is also optimized: the classical Young [15]–Daly [16] formulas minimize expected wasted work from MTBF and checkpoint cost, while adaptive variants track fluctuating MTBF on dynamically provisioned, multi-tenant resources [17]. These mechanisms reduce resilience cost at scale, but still presuppose the manual instrumentation step we automate: someone must decide what to checkpoint and where.

By contrast, transparent checkpointing tools such as BLCR [18], DMTCP [19], and CRIU [20] reduce developer effort by capturing process state automatically, but often include caches, transient buffers, and rebuildable state. This can produce oversized checkpoints, limit scalability under elasticity, and complicate portability across heterogeneous nodes. Compiler-assisted approaches [21] narrow the snapshot through static analysis, but require per-language passes and can struggle with the templated, dynamically allocated, irregularly accessed data structures common in modern HPC and AI codes. Although transparent and compiler-assisted approaches can benefit from the same multi-level optimizations, they do not provide the combination we target: low-effort automation with application-level checkpoint efficiency.

To our knowledge, no prior work has attempted to study frontier LLM code generation to the point where it allows end-to-end automation (i.e., analyze the application code and gen-

TABLE I
BENCHMARK APPLICATIONS ANALYZED IN THIS PAPER.

App	Lang	Checkpoint mech.	LOC
Athena++	C++	native	132K
CLAMR	C++	native	81K
CoMD	C	POSIX	5.6K
HPCG	C++	POSIX	6.2K
HyPar	C	native	63K
LAMMPS	C++	native	627K
Nyx	C++	AMReX-native	441K
OpenLB	C++	native	338K
PRK_Stencil	C	POSIX	1.0K
QMCPACK	C++	native	532K
ROSS	C	RIO	15K
SAMRAI	C++	HDF5	598K
Smilei	C++	HDF5	153K
SPARTA	C++	native	174K
SPPARKS	C++	native	52K
SW4lite	C++	native	57K
WarpX	C++	AMReX-native	553K

erate application-level checkpointing that captures a minimal critical state), thus combining the advantages of transparent checkpointing with application-level checkpointing.

III. RESILIENT BENCHMARK SUITES

To test the effectiveness of frontier LLM models with respect to automated resilience, we assembled a benchmark suite of unmodified MPI scientific applications drawn from a wide spectrum of HPC domains. Three selection criteria apply: (i) each application must be open source and reproducible from its upstream repository; (ii) each must ship with a native checkpoint/restart implementation we can use as the upstream-reference baseline; and (iii) together the suite must cover diverse data-structure shapes (fixed-size arrays, variable-per-rank particle pools, adaptive meshes), per-step synchronization patterns, and checkpoint back-ends.

Table I lists the seventeen applications that meet these criteria. They span computational domains from sparse linear solvers (HPCG [22]) and stencil kernels (PRK_Stencil) to molecular dynamics (CoMD [23], LAMMPS [24]), lattice Boltzmann fluid dynamics (OpenLB [25]), direct-simulation Monte Carlo (SPARTA [26]), parallel discrete-event simulation (ROSS), kinetic Monte Carlo (SPPARKS), Quantum Monte Carlo (QMCPACK), particle-in-cell plasma physics (Smilei, WarpX), seismic wave propagation (SW4lite), shock hydrodynamics (CLAMR), conservation-law solvers (HyPar), AMR cosmology (Nyx), AMR astrophysics (Athena++ [27]), and AMR mesh frameworks (SAMRAI). The suite mixes C and C++ codebases, fixed-size and variable-per-rank state, and POSIX, native, AMReX-native, RIO, and HDF5 checkpoint back-ends, exercising the principal checkpoint patterns encountered in production MPI software.

IV. EXPERIMENT SETUP

Experimental Pipeline. Figure 1 sketches the end-to-end pipeline we use to test whether a frontier LLM can au-

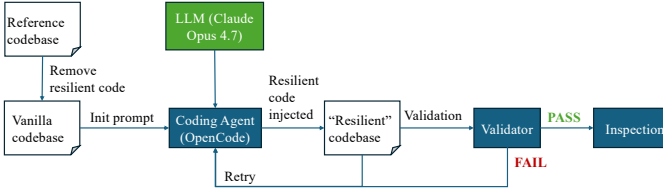


Fig. 1. End-to-end LLM-driven pipeline for adding checkpoint/restart resilience to an unmodified MPI application.

onomously add checkpoint/restart resilience to an MPI scientific code. We start by stripping the upstream source of any existing resilience logic to produce a *vanilla codebase*; this is the only artifact the LLM ever sees. We then drive an iterative code-generation loop, in which each iteration executes two phases in sequence with no human in the loop.

- 1) **Code Generation.** We give Anthropic’s Claude Opus 4.7 [28], invoked through the `opencode` coding-agent CLI [3], the vanilla source tree plus a fixed instruction prompt that asks it to use VeloC [4] (a multi-level checkpoint/restart runtime designed for HPC) to protect the application against process failure. The agent issues `read/edit` tool calls on the source tree and writes a modified copy we call the *resilient codebase*, which must preserve the vanilla code’s functional behavior while adding fault tolerance. The prompt also asks the agent to log its reasoning, intended action, observed result, and next step at every iteration, giving us a per-step transcript of the transformation.
- 2) **Validation.** An independent validator builds the resilient codebase and, if the build succeeds, runs it under two scenarios. *Failure-free execution* confirms the modified code still produces output equivalent to the vanilla under tolerance, ensuring the functional contract is intact. *Failure-injected execution* kills one rank mid-run, re-launches the binary, and checks four conditions: (i) at least one checkpoint file was written before the kill; (ii) the harness actually delivered the failure (no spurious early exit); (iii) the post-restart output matches the vanilla, confirming the checkpoint plus recovery path reproduces the intended results; and (iv) end-to-end wall time stays meaningfully below a full restart-from-scratch, evidence the checkpoint cadence is operationally useful.

If any check fails or the run times out, the loop iterates: the agent receives the original prompt, its previous resilient code, and the validator’s structured feedback (build log, execution stdout/stderr, per-condition pass/fail flags) and is asked to produce a corrected version. Each application is allowed up to ten iterations; if the validator passes before that cap, the resulting code is promoted to the inspection phase analyzed in §V.

Metrics and Configurations. The inspection phase collects four metrics per application: (i) *Code generation time*: cumulative wall time of the LLM/validator loop, including both the agent’s thinking time and the validator’s build/run time;

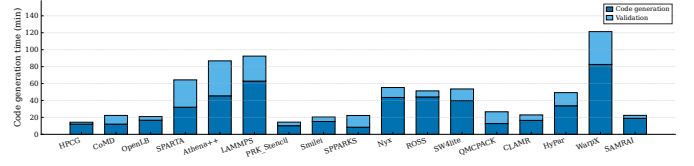


Fig. 2. Time consumed by the LLM iterative loop to produce a working checkpoint/restart implementation, per app.

(ii) *Total tokens*: input plus output tokens consumed across all iterations, a direct proxy for API spend; (iii) *End-to-end execution time*: wall time of one application run under each scenario, quantifying both instrumentation overhead and recovery cost; and (iv) *Per-frame checkpoint footprint*: bytes written by a single checkpoint event, capturing the storage cost of the chosen state coverage. All measurements were collected on a single development host with 4 MPI ranks per application. Per-app input arguments were tuned so one failure-free run takes 60–200 s, short enough to keep the iterative loop tractable yet long enough for failure injection to produce meaningful checkpoint, recovery, and output signals. For every (application, scenario) cell we run three independent trials and report the mean. All runs use the latest released version of VeloC [4].

V. KEY FINDINGS

We trace what actually happens inside the iterative loop, distilled from the per-iteration `opencode_stdout` transcripts that record the agent’s reasoning and actions step by step. Across all seventeen applications the transcripts reveal the *same four-phase procedure*, applied with very few app-specific deviations:

- 1) **Reconnaissance.** The agent locates the application’s main timestep loop, starting from the entry-point file (`main.cpp` or equivalent) and following function calls inward. It also locates the VeloC installation on disk and reads the VeloC documentation to learn the API it will need.
- 2) **Critical-state identification.** The agent walks the application’s central data structure field by field and labels each field as either *rebuildable* (the input deck plus the application’s own initialization code can recreate it) or *evolving* (the value changes inside the timestep loop and therefore must be saved).
- 3) **Implementation.** The agent wires VeloC into the application: it initializes the runtime at startup, declares memory regions of evolving fields to checkpoint, calls the checkpoint primitive at a chosen cadence inside the timestep loop, and adds a startup branch that detects an existing checkpoint, restores the saved state, and resumes the loop from the saved counter.
- 4) **Self-review.** The agent walks the change diff and lists the corner cases it has thought through (first-ever run, partial checkpoint write, restart-counter collisions, halo cells, RNG state, and so on), justifying how each is handled before declaring the iteration ready for validation.

This procedure successfully transformed *all* seventeen vanilla codebases into resilient ones. We walk through each application below in roughly increasing order of state complexity, then summarize the cross-cutting design choices that recur across the suite.

a) *HPCG (2 iterations to PASS)*: HPCG is the suite’s simplest case: each CG solve starts from the same zeroed initial vector and no state carries over between solves, so the only critical state is a counter plus the residual array. The agent rejected saving the sparse matrix, multigrid hierarchy, and per-solve scratch vectors as deterministically rebuildable from the input dimensions.

b) *CoMD (2 iterations to PASS)*: CoMD is harder than HPCG because state evolves inside the timestep loop: atoms migrate between MPI subdomains every step, so the per-rank atom list (positions, momenta, forces, identifiers) plus a small link-cell bookkeeping array changes continuously. Everything else (simulation box, species table, potentials, link-cell geometry) is fixed at startup, and halo cells are repopulated on the first post-restart timestep by CoMD’s own halo-exchange. The agent saved the atom arrays plus the loop counter and skipped halos.

c) *OpenLB (1 iteration to PASS)*: OpenLB stores far more data per process than CoMD, and that data is tangled inside an internal object whose layout would be tedious and error-prone to list by hand. Conveniently, OpenLB already provides three helper methods designed for sending the object between processes (return its size, pack it into a buffer, unpack it from a buffer); after an intensive search over the codebase, the agent found and reused them as a black-box serialization, so it never needed to know what the object actually contains.

d) *SPARTA (2 iterations to PASS)*: SPARTA introduces a new wrinkle: per-rank particle counts vary because particles drift between ranks every timestep, ruling out fixed-size memory regions. The agent used VeloC’s file-based API to write a small structured payload per rank (timestep counter, simulation-time scalars, particle count, particle array, RNG state) and skipped the static grid topology, species tables, and master RNG.

e) *Athena++ (4 iterations to PASS)*: Athena++ raises the stakes again: its state lives in an evolving AMR mesh tree plus a long list of conditional physics arrays, so field-by-field enumeration would have been both labor-intensive and high-risk. The agent instead implemented a writer/reader pair dedicated to Athena++’s mesh format to dump and reconstruct the critical state from disk.

f) *LAMMPS (7 iterations to PASS)*: LAMMPS has the most complex state in the suite. Its atoms move between ranks as in SPARTA, but it also tracks two things no other app does: bonds and angles that link atoms together (a graph the checkpoint must keep consistent after migration), and many small pieces of state owned by whichever optional physics modules the user turns on (each one keeps its own counters and random-number state).

g) *PRK_Stencil (2 iterations to PASS)*: PRK_Stencil resembles HPCG in spirit but evolves real per-step state: a 2D

stencil sweeps a single working array, so the only critical state is the iteration counter and the array itself. Iteration 1 wired the `VELOC_Mem_protect` calls but failed validation because `veloc.cfg` sat under `Stencil/` where the build system did not stage it; iteration 2 was a one-line config relocation to the project root.

h) *Smilei (3 iterations to PASS)*: Smilei is a relativistic PIC code with a deeply nested particle/field hierarchy, so the agent declined to enumerate it field by field. Instead it dumped the three integrators that drive the loop (`itime`, `time_prim`, `time_dual`) as a small fixed-size byte payload through VeloC’s file-based API and let Smilei’s own initialization rebuild the rest. Iterations 2 and 3 were build-system fixes: missing `-lveloc` in the C++ link line surfaced as undefined-symbol errors that took two passes to localize to the right `LDFLAGS` block in the bespoke makefile.

i) *SPPARKS (2 iterations to PASS)*: SPPARKS is closer to LAMMPS than to HPCG: a rejection-kinetic-Monte-Carlo lattice solver carries a non-trivial mix of evolving state (current time, per-site spin array, accept/reject counters, Marsaglia RNG state). The agent registered each component with `VELOC_Mem_protect` and explicitly excluded the static lattice topology and the master RNG seed as deterministically rebuildable. Iteration 2 was a config-placement fix mirroring the `PRK_Stencil` pattern.

j) *Nyx (5 iterations to PASS)*: Nyx is the first AMReX-based app in the suite and exposes the difficulty of integrating with a heavy C++ template framework: critical state lives in distributed `MultiFab` objects whose extents change with the AMR hierarchy. The agent chose memory-protect over a coordinator approach, writing helper functions that walk each `MultiFab` and register its underlying float arrays with VeloC. The five-iteration arc was almost entirely build-system pain: header-include order, AMReX namespace resolution, and CMake integration each cost an iteration before the resilient binary linked cleanly.

k) *ROSS (3 iterations to PASS)*: ROSS is a parallel discrete-event simulator whose recovery semantics differ from every other app in the suite: after restart the global virtual time (GVT) machinery must not double-count events committed before the failure. The agent serialized a small POD struct (current GVT, per-rank event counters, anti-message bookkeeping) through VeloC’s file-based API and added a post-restore correction step that re-establishes the cumulative-statistics baseline. Iterations 2 and 3 closed two related bugs in that correction path that surfaced as biased steady-state event-rate measurements.

l) *SW4lite (2 iterations to PASS)*: SW4lite is a 4th-order seismic wave-propagation kernel whose state is three rolling time-step buffers (`Um`, `U`, `Up`) plus the step counter. The agent dumped the three buffers and the counter as a single opaque payload via VeloC’s file-based API rather than registering each array individually, because the buffers are allocated as flat C arrays with sizes already known to the host code. Iteration 2 was a CMake placement fix: the agent had added VeloC linkage to the wrong `add_executable` target.

m) *QMCPACK (3 iterations to PASS)*: QMCPACK runs Monte Carlo electronic-structure samples whose physics state is owned by an internal walker manager that the agent judged too risky to dissect; instead it preserved a minimal coordinator (the QMC iteration counter via `VELOC_Mem_protect`) plus a small file-based blob holding the random-number-generator state. This is why QMCPACK’s per-frame checkpoint is only 40 bytes in Figure 5: physics state is regenerated by QMCPACK’s own initialization rather than carried across the recovery boundary. Iteration 3 fixed a comparator that had been keying on a banner-style `completed successfully` string; the agent rewrote it to compare the recovered reference energy within tolerance 0.01.

n) *CLAMR (3 iterations to PASS)*: CLAMR is an adaptive-mesh shallow-water hydro code where each rank owns a different cell count, so fixed-size memory regions were ruled out. The agent dumped per-rank arrays `H` (height), `U`, `V` (momenta), and the active cell count through VeloC’s file-based API. Iterations 2 and 3 resolved two cascading restore-time bugs: a per-rank size mismatch when ranks resized between checkpoints, and a conflict with CLAMR’s L7 communication library, which holds MPI state that had to be re-initialized after the VeloC restore call.

o) *HyPar (3 iterations to PASS)*: HyPar is a finite-difference hyperbolic-parabolic solver whose critical state is the conserved-variable array `u[]` plus integrator scratch buffers and the step counter; the agent registered each via `VELOC_Mem_protect` following the HPCG/CoMD template. Iterations 2 and 3 were build and runtime configuration fixes: integrating VeloC into HyPar’s CMake graph took one iteration, and a `getenv`-based runtime path lookup that returned NULL inside the validator’s process took another to diagnose and replace with an explicit config path.

p) *WarpX (5 iterations to PASS)*: WarpX is the largest codebase in the suite (627K LOC, AMReX-based PIC) and ships full native checkpoint/restart through the `amr.restart=<chkdir>` input parameter. The agent first attempted a memory-protect approach mirroring Nyx, registering top-level `MultiFab` buffers, but iteration 3 logs show it abandoned that path after concluding that capturing all conditional physics modules (electromagnetic solvers, particle species, embedded boundaries) would be intractable. The pivot was decisive: VeloC keeps only the step counter and the native checkpoint directory name (24 bytes per frame in Figure 5), and on restart the agent’s shim rewrites the `amr.restart` input and re-execs WarpX through its own restart entry point. Iterations 4 and 5 hardened that shim against partially-written checkpoint directories.

q) *SAMRAI (1 iteration to PASS)*: SAMRAI converged in a single iteration because it ships a first-class `tbox::RestartManager` infrastructure that already serializes every registered `Database` object to disk. The agent detected this immediately and selected the same coordinator pattern as WarpX: VeloC tracks only the step counter and the `RestartManager`’s output directory (304 bytes per frame), and on restart hands control to

`RestartManager::openRestartFile` rather than reconstructing patch state itself. SAMRAI is the strongest evidence in the suite that when an app’s own restart abstractions are clean and discoverable, the agent recognizes them on the first pass and writes minimal coordinator code rather than reimplementing them.

r) *Cross-cutting observations*: Three patterns recur across all seventeen apps. First, the agent always starts from a *small* state-coverage set rather than a conservative *everything* one: it picks the few fields that actually evolve, defends every rejection in writing, and only adds coverage if validator feedback forces it. This is the proximate reason failure-free overhead is essentially zero across the suite. Second, the agent’s willingness to *reuse application-specific infrastructure* (OpenLB’s `serialize` methods, Athena++’s restart-file reader, WarpX’s `amr.restart`, SAMRAI’s `RestartManager`, LAMMPS’s `restart/read_restart` commands) is the single biggest predictor of fast convergence: nine of the seventeen apps converge in ≤ 3 iterations because the agent found and reused such infrastructure rather than re-implementing it. Third, when the validator reports a failure the agent treats it as structured triage: read the exact error, form one hypothesis, investigate it before forming the next, and only then propose a fix. Failures cluster into three causes: configuration misplacement (e.g., `veloc.cfg` not at the path the validator probes; fixed in 1–2 iterations on PRK_Stencil, SPPARKS, HyPar, QMCPACK), build/CMake integration issues (Athena++, SW4lite, HyPar, Nyx), and post-restart determinism drift (LAMMPS atom-owner ordering, which required six hypothesis-elimination iterations).

Time to Construct Resilient Code. The iterative loop consumed 12.7 hours total across the seventeen apps, averaging 45 minutes per app, with a $\sim 8.5\times$ spread between the fastest (HPCG, 14 min) and the slowest (WarpX, 121 min). Figure 2 decomposes per-app loop time into LLM code generation (dark blue, bottom) and validation (light blue, top): *code generation dominates on every app*, taking a median of 61% of the total (range 50–84%).

The bar shapes track the difficulty ordering. Small fixed-state apps (HPCG 14 min, CoMD 22 min, OpenLB 21 min) sit at the bottom because their critical state is small and the iteration 1 design was structurally correct, with at most a one-line operational fix on iteration 2. Medium-complexity apps (SPARTA 64 min, Athena++ 87 min, LAMMPS 92 min) require multiple iterations to capture richer state surfaces. The largest codebases (WarpX 121 min, Nyx 55 min, ROSS 51 min, SW4lite 54 min) dominate the chart because either their per-rank state is intricate or recovery semantics required multi-iteration debugging.

Token Cost. Total tokens across the seventeen apps sum to 86.1 M (average 5.1 M per app), corresponding to roughly \$200–1500 in API spend at current frontier-model rates [29]. Figure 3 spans $\sim 85\times$, from CoMD’s 0.28 M up to WarpX’s 23.7 M.

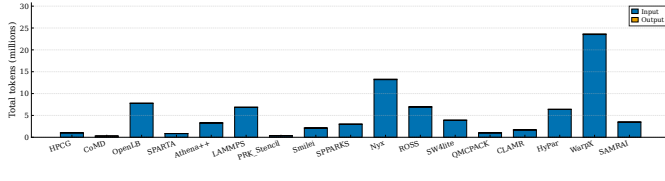


Fig. 3. Total Claude Opus 4.7 tokens consumed by the iterative loop, per app.

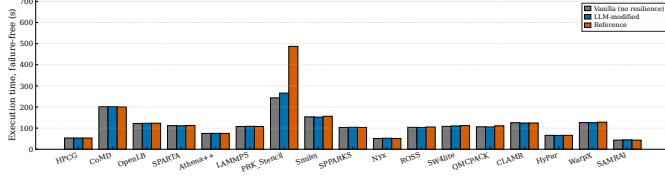


Fig. 4. Per-app average execution time under the failure-free scenario.

Token cost correlates only loosely with code generation time. WarpX tops the chart at 23.7 M because its 553K-line PIC+AMR codebase requires the agent to scan a substantial portion of the source to find the relevant integrator, particle, and field hooks. OpenLB at 7.8 M converged in a single iteration but still needed extensive scanning of its 338K-line source tree to discover the built-in serialize methods. CoMD sits at the bottom (0.28 M) because of its small 5.6K-LOC codebase and small critical state. The token total is dominated by source-tree size, not by iteration count.

The stacked decomposition shows input tokens account for roughly 99% of every bar (the agent reads orders of magnitude more text than it writes): *input-context size is the dominant token cost driver*, so pre-summarizing or chunking large source trees would translate directly into spend reductions, especially on the largest apps (WarpX, QMCPACK 532K, SAMRAI 598K, Nyx 441K) where exploration substantially exceeds eventual edits.

Overhead: Runtime and Storage. Figure 4 shows an important result: *the LLM-generated checkpoint instrumentation imposes essentially zero failure-free overhead*. Fourteen of seventeen apps fall within $\pm 2\%$ of vanilla wall time (range -1.1% for CLAMR to $+1.9\%$ for SW4lite), well inside measurement noise. Two apps fall in the moderate band (Nyx $+2.9\%$, SAMRAI $+2.8\%$). Only PRK_Stencil shows a larger overhead at $+9.2\%$, attributable to the chosen checkpoint cadence on its tight stencil loop. The LLM bars track the upstream reference closely on every app, so the generated code matches a human-engineered native implementation.

Figure 5 shows full-checkpoint sizes spanning many orders of magnitude, from per-frame footprints of tens of bytes up to hundreds of megabytes (CoMD’s 194 MB full per-rank atom array; PRK_Stencil’s 257 MB grid). The smallest LLM checkpoints (24 B WarpX, 256 B Smilei, 284 B Nyx, 304 B SAMRAI, 312 B ROSS, 420 B QMCPACK, 744 B HyPar) reflect the *coordinator pattern* described in §??: on these apps the LLM defers physics-state preservation to the

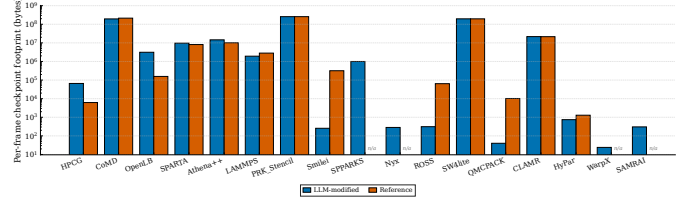


Fig. 5. Payload size of a full recoverable checkpoint.

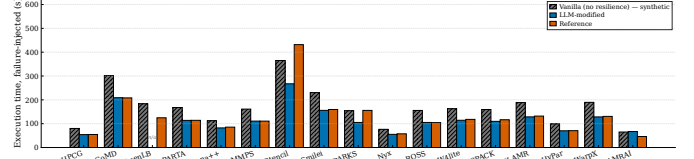


Fig. 6. Per-app end-to-end execution time with a failure injected mid-run.

application’s native checkpoint mechanism and writes only a step counter plus the native-checkpoint directory pointer through VeloC, which is sufficient for VeloC to coordinate the restart. LLM and reference agree on the order of magnitude on most apps; where they diverge (e.g. HPCG LLM 66 KB vs. reference 6 KB; OpenLB LLM 3 MB vs. reference 157 KB) the LLM is more conservative, persisting diagnostic accumulators the reference recomputes on restart. **[TODO: 4 of 17 reference checkpoints (Nyx, SAMRAI, SPPARKS, WarpX) cannot be measured because their upstream native checkpoint formats (AMReX-native, HDF5, custom binary) are not introspected by our metric collector; bars for those references render as “n/a”. Resolution: extend the metric collector with format-specific introspection.]**

Resilience. Figure 6 shows per-app end-to-end runtime when the validator kills one MPI rank mid-run, then restarts the app. The LLM runtime sits close to the reference on most apps, with recovery ratios (once / failure-free) clustering tightly between $1.01\times$ and $1.09\times$ for fourteen of seventeen apps. Both the LLM and reference recover within roughly one third of the wall-clock penalty paid by the vanilla baseline, which has no checkpoint protection and must restart from scratch. Two outliers warrant note: SAMRAI (LLM ratio $1.50\times$) and SPPARKS (reference ratio $1.51\times$) effectively restart-from-scratch because their LLM checkpoint preserves only meta-data (SAMRAI) or because the upstream code lacks native checkpoint support (SPPARKS). **[TODO: OpenLB’s LLM once-cell summary in the chart is computed from a stale $n=3$ aggregate (mean 205 s) that does not include the two clean post-extension runs (true mean 173 s, median 126 s); re-aggregate the OpenLB_baseline summary block from the $n=5$ raw runs before camera-ready.]** Apart from these, the chart is operational evidence the LLM-generated code does real recovery work: detecting checkpoint files, restoring state, and resuming with overhead bounded by the checkpoint cadence rather than by the size of the original computation.

VI. CONCLUSION AND FUTURE WORK

This paper investigates whether frontier LLM coding agents can automate end-to-end checkpoint/restart integration for existing MPI scientific applications. Across seventeen applications with diverse domains (1K to 627K LOC), data structures, and checkpointing patterns, the agent generated working resilient implementations without human intervention after the initial prompt. The generated code achieved near-zero failure-free overhead on fourteen of seventeen applications, recovered from injected failures with performance close to upstream native implementations on every app where the comparison is available, and converged in under two hours per application. These results suggest that LLM-driven resilience engineering can substantially reduce the expert effort required to add production-grade fault tolerance to a meaningful subset of HPC applications.

Future work should improve the robustness, scalability, and usability of this approach. Specialized pipelines with resilience-specific skills, source-code summarization, checkpoint-state analysis, and targeted validation feedback could reduce cost and improve reliability. Broader evaluations are needed for repeated failures, realistic I/O environments, multi-node executions, and applications with more irregular or distributed state. Finally, generated checkpoint code should become easier to audit, maintain, and integrate into production workflows so that automated resilience generation can be used safely in practice.

REFERENCES

- [1] F. Cappello, A. Geist, W. Gropp, S. Kale, B. Kramer, and M. Snir, "Toward exascale resilience: 2014 update," *Supercomputing Frontiers and Innovations: An International Journal*, vol. 1, no. 1, pp. 5–28, 2014.
- [2] B. Nicolae, J. Wozniak, T. Bicer, H. Nguyen, P. Patel, H. Pan, A. Gueroudji, M. Gonthier, V. Hayot-Sasson, E. Huerta, K. Chard, R. Chard, M. Dorier, N. S. V. Rao, A. Al-Najjar, A. Corsi, and I. Foster, "Diaspora: Resilience-enabling services for real-time distributed workflows," in *NRDPISI'24: The 1st International Workshop on Near Real-time Data Processing for Interconnected Scientific Instruments (co-located with eScience'24)*, Osaka, Japan, 2024. [Online]. Available: <https://hal.inria.fr/hal-04819775>
- [3] "OpenCode: Open-source ai coding agent," <https://github.com/anthropics/opencode>, 2024.
- [4] B. Nicolae, A. Moody, E. Gonsiorowski, K. Mohror, and F. Cappello, "VeloC: Towards high performance adaptive asynchronous checkpointing at large scale," in *IPDPS'19: The 2019 IEEE International Parallel and Distributed Processing Symposium*, Rio de Janeiro, Brazil, 2019, pp. 911–920. [Online]. Available: <https://hal.inria.fr/hal-02184203>
- [5] L. Bautista-Gomez, S. Tsuboi, D. Komatitsch *et al.*, "FTI: High performance fault tolerance interface for hybrid systems," *ACM/IEEE International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2011.
- [6] A. Moody, G. Bronevetsky, K. Mohror, and B. R. de Supinski, "Scalable checkpoint/restart for MPI," in *ACM/IEEE International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2010.
- [7] A. Maurya, B. Nicolae, M. Rafique, T. Tonellot, and F. Cappello, "Towards Efficient I/O Scheduling for Collaborative Multi-Level Checkpointing," in *MASCOTS'21: The 29th IEEE International Symposium on the Modeling, Analysis, and Simulation of Computer and Telecommunication Systems*, Virtual, Portugal, 2021, pp. 1–8. [Online]. Available: <https://hal.inria.fr/hal-03344362>
- [8] A. Maurya, R. Underwood, M. Rafique, F. Cappello, and B. Nicolae, "DataStates-LLM: Lazy asynchronous checkpointing for large language models," in *HPDC'24: The 33rd International Symposium on High-Performance Parallel and Distributed Computing*, Pisa, Italy, 2024. [Online]. Available: <https://hal.science/hal-04614247>
- [9] A. Maurya, M. Rafique, T. Tonellot, H. AlSalem, F. Cappello, and B. Nicolae, "GPU-enabled asynchronous multi-level checkpoint caching and prefetching," in *HPDC'23: The 32nd International Symposium on High-Performance Parallel and Distributed Computing*, Orlando, USA, 2023, pp. 73–85. [Online]. Available: <https://hal.inria.fr/hal-04119928>
- [10] M. J. Gossman, B. Nicolae, and J. C. Calhoun, "Scalable I/O aggregation for asynchronous multi-level checkpointing," *Future Generation Computer Systems*, vol. 160, pp. 420–432, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167739X24002929>
- [11] B. Nicolae and F. Cappello, "AI-Ckpt: Leveraging memory access patterns for adaptive asynchronous incremental checkpointing," in *HPDC'13: 22th International ACM Symposium on High-Performance Parallel and Distributed Computing*, New York, USA, 2013, pp. 155–166. [Online]. Available: <https://hal.archives-ouvertes.fr/hal-00809847/en>
- [12] N. Tan, J. Luettgau, J. Marquez, K. Terianishi, N. Morales, S. Bhowmick, F. Cappello, M. Taufer, and B. Nicolae, "Scalable incremental checkpointing using GPU-accelerated de-duplication," in *ICPP'23: The 52nd International Conference on Parallel Processing*, Salt Lake City, USA, 2023, pp. 665–674. [Online]. Available: <https://hal.inria.fr/hal-04173764>
- [13] X. Liang, S. Di, D. Tao, S. Li, B. Nicolae, Z. Chen, and F. Cappello, "Improving performance of data dumping with lossy compression for scientific simulation," in *CLUSTER'19: IEEE International Conference on Cluster Computing*, Albuquerque, USA, 2019, pp. 1–11.
- [14] A. Maurya, M. M. Rafique, F. Cappello, and B. Nicolae, "Towards efficient I/O pipelines using accumulated compression," in *HIPC'23: 30th IEEE International Conference on High Performance Computing, Data, and Analytics*, Goa, India, 2023. [Online]. Available: <https://hal.inria.fr/hal-04343670>
- [15] J. W. Young, "A first order approximation to the optimum checkpoint interval," *Communications of the ACM*, vol. 17, no. 9, pp. 530–531, 1974.
- [16] J. T. Daly, "A higher order estimate of the optimum checkpoint interval for restart dumps," *Future Generation Computer Systems*, vol. 22, no. 3, pp. 303–312, 2006.
- [17] T. Dey, K. Sato, B. Nicolae, J. Guo, J. Domke, W. Yu, F. Cappello, and K. Mohror, "Optimizing asynchronous multi-level checkpoint/restart configurations with machine learning," in *HPS'20: The 2020 IEEE International Workshop on High-Performance Storage*, New Orleans, USA, 2020, pp. 1036–1043. [Online]. Available: <https://hal.archives-ouvertes.fr/hal-02914478>
- [18] P. H. Hargrove and J. C. Duell, "Berkeley Lab Checkpoint/Restart (BLCR) for Linux clusters," *Journal of Physics: Conference Series (SciDAC)*, vol. 46, pp. 494–499, 2006.
- [19] J. Ansel, K. Arya, and G. Cooperman, "DMTCP: Transparent checkpointing for cluster computations and the desktop," in *IEEE International Symposium on Parallel and Distributed Processing (IPDPS)*, 2009.
- [20] The CRIU Project, "CRIU: Checkpoint/restore in userspace," <https://criu.org>, 2024.
- [21] G. Bronevetsky, D. Marques, K. Pingali, and P. Stodghill, "Automated application-level checkpointing of MPI programs," in *ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, 2003, pp. 84–94.
- [22] J. Dongarra, M. A. Heroux, and P. Luszczek, "HPCG: A new benchmark for high-performance computing systems," *International Journal of High Performance Computing Applications*, vol. 30, no. 1, pp. 3–10, 2016.
- [23] Exascale Co-Design Center for Materials in Extreme Environments, "CoMD: Classical molecular dynamics proxy application," <https://github.com/ECP-copa/CoMD>, 2017.
- [24] S. Plimpton, "Fast parallel algorithms for short-range molecular dynamics," *Journal of Computational Physics*, vol. 117, no. 1, pp. 1–19, 1995.
- [25] M. J. Krause, A. Kummerländer, S. J. Avis, H. Kusumaatmaja, D. Dapelo, F. Klemens, M. Gaedtker, N. Hafen, A. Mink, R. Trunk *et al.*, "OpenLB: Open source lattice Boltzmann code," *Computers & Mathematics with Applications*, vol. 81, pp. 258–288, 2021.
- [26] S. J. Plimpton, S. G. Moore, A. Borner, A. K. Stagg, T. P. Koehler, J. R. Torczynski, and M. A. Gallis, "Direct simulation Monte Carlo on

petaflop supercomputers and beyond,” *Physics of Fluids*, vol. 31, no. 8, p. 086101, 2019.

- [27] J. M. Stone, K. Tomida, C. J. White, and K. G. Felker, “The Athena++ adaptive mesh refinement framework: Design and magnetohydrodynamic solvers,” *The Astrophysical Journal Supplement Series*, vol. 249, no. 1, p. 4, 2020.
- [28] Anthropic, “Claude: AI assistant by Anthropic,” <https://www.anthropic.com/claude>, 2024.
- [29] —, “Pricing,” <https://www.anthropic.com/pricing>, 2025, accessed: 2026-05-04.